TAREA 2 · BASES DE DATOS NOSQL

EventFlow

Venta de entradas con microservicios, tres bases de datos, SAGA y Chain of Responsibility

FastAPI MongoDB Redis PostgreSQL Docker Compose

Pre-defensa · 5 de noviembre de 2026

EL PROBLEMA

Dos exigencias opuestas en el mismo sistema

Lecturas

Catálogo y perfiles

- Muchas consultas
- Prioridad: **disponibilidad**
- Se acepta un dato algo viejo

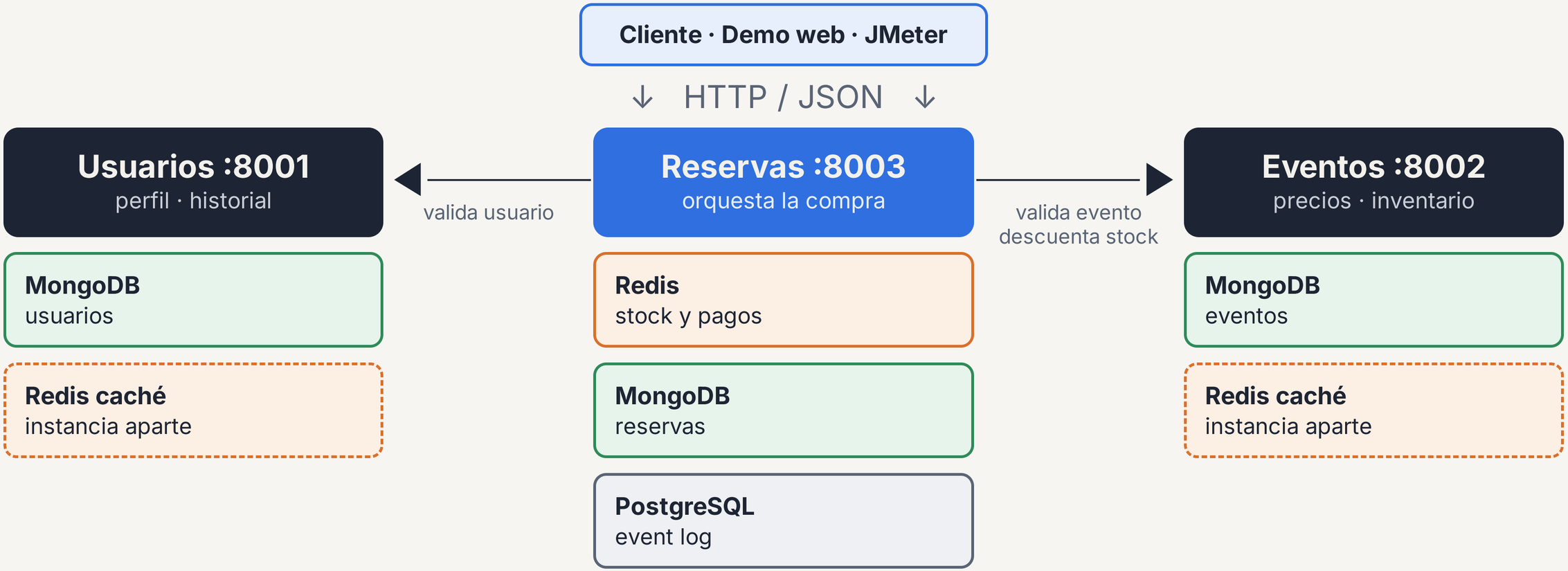
Compras

Reservas y pagos

- Muchos compradores, mismo stock
- Prioridad: **consistencia fuerte**
- Nunca vender de más ni cobrar dos veces

ARQUITECTURA

Tres microservicios, tres bases



Dos Redis separados: la caída de la caché no frena las compras y la de pagos no frena las lecturas · entre servicios: HTTP con reintentos y circuit breaker

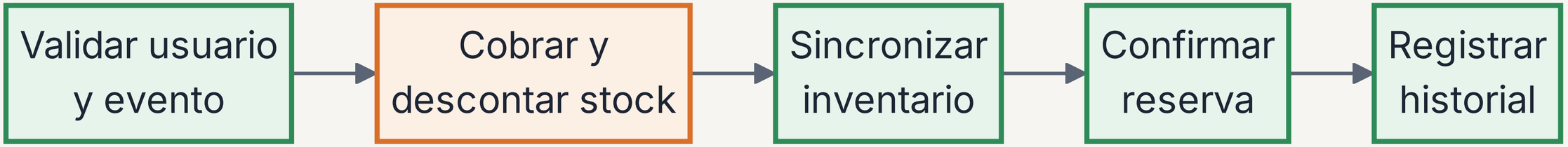
SELECCIÓN DE BASES

Cada base, por lo que necesita su dato

Base	Qué guarda	Por qué
MongoDB documental · NoSQL	Usuarios, eventos y reservas	Un documento trae todo junto, sin joins
Redis clave-valor · NoSQL	Stock y pagos; otra instancia aparte para la caché	Un script Lua atómico decide la venta en < 1 ms
PostgreSQL relacional	Event log de cada compra	SQL y vistas para auditar y analizar

ESTRATEGIA DE TRES BASES

Una compra pasa por las tres



MongoDB

Guarda el negocio

Redis

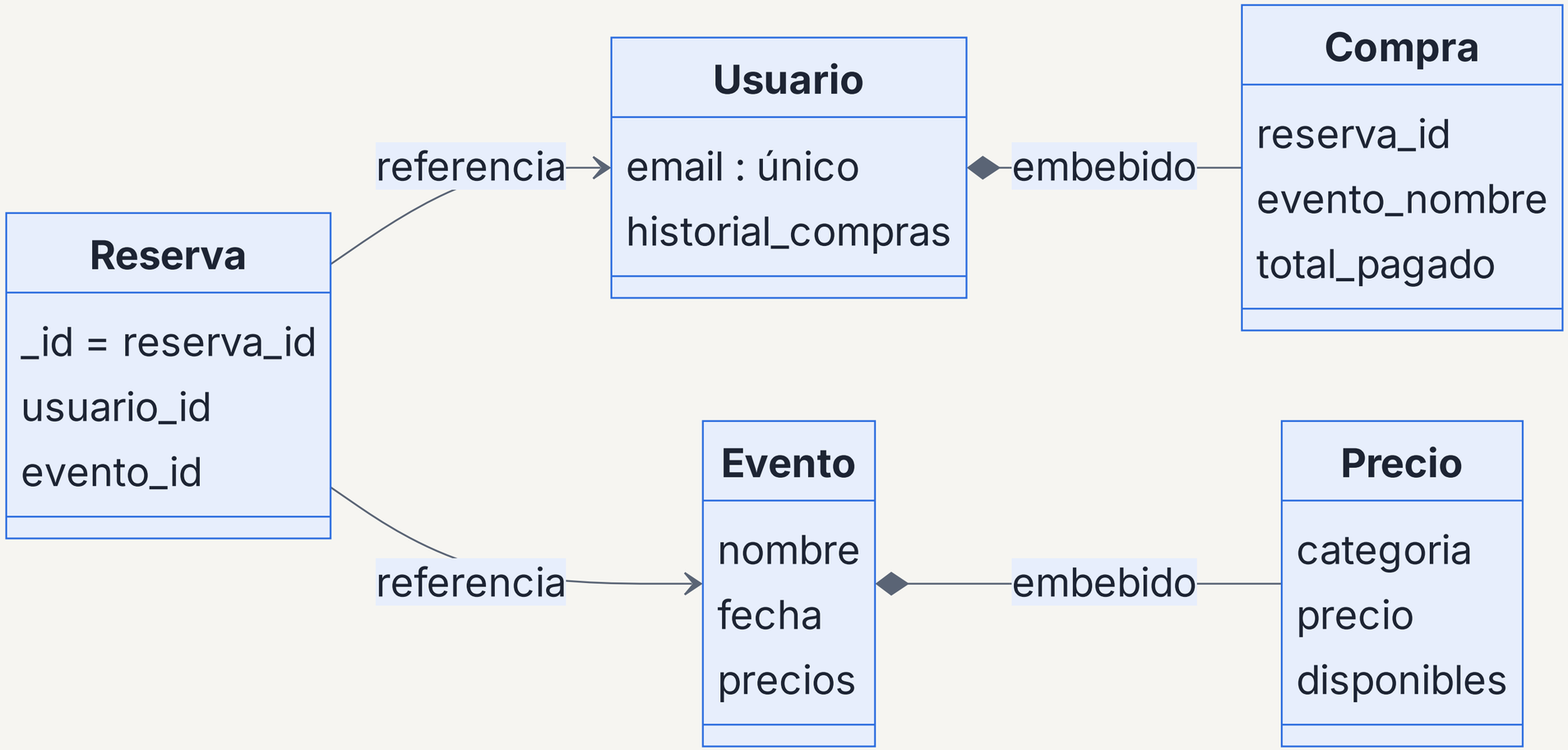
Decide la venta

PostgreSQL

Registra cada paso

Cada paso también escribe su evento en PostgreSQL, sin bloquear la compra

Embebido dentro de un servicio, referencia entre servicios



Embebido: se lee siempre junto · Referencia: vive en otro servicio

CONSISTENCIA POR OPERACIÓN

Lecturas disponibles, compras consistentes

AP

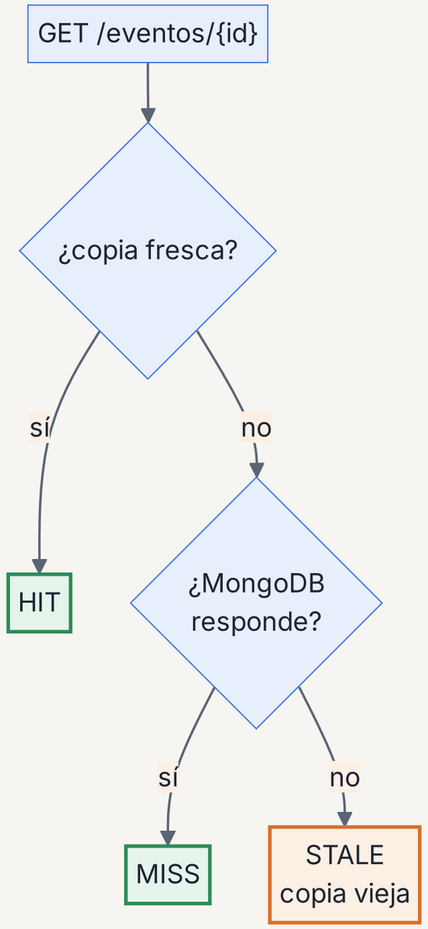
Leer usuario o evento

Caché Redis: si MongoDB cae, responde con la última copia.

CP

Comprar

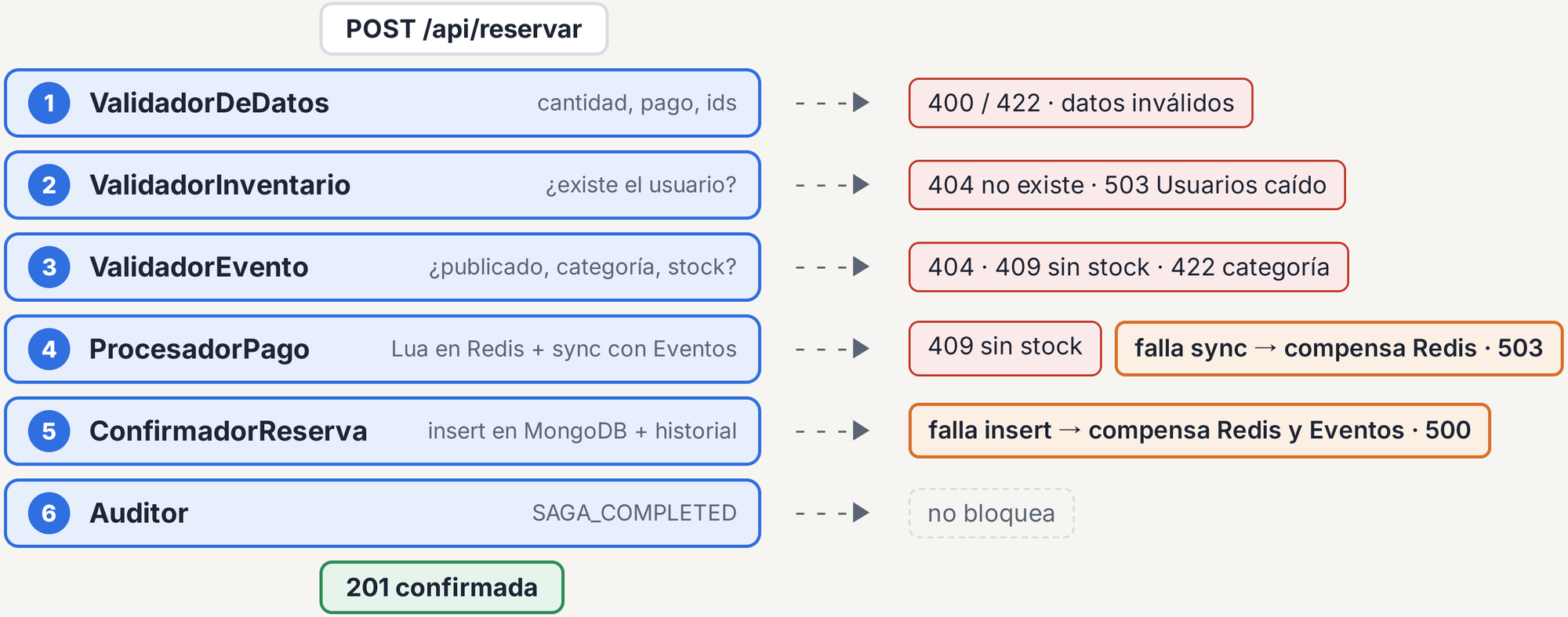
Lua atómico + idempotencia + SAGA. Si no puede garantizarlo, responde error.



Cada base corre en un solo nodo: la disponibilidad se demuestra con la caché

CHAIN OF RESPONSIBILITY

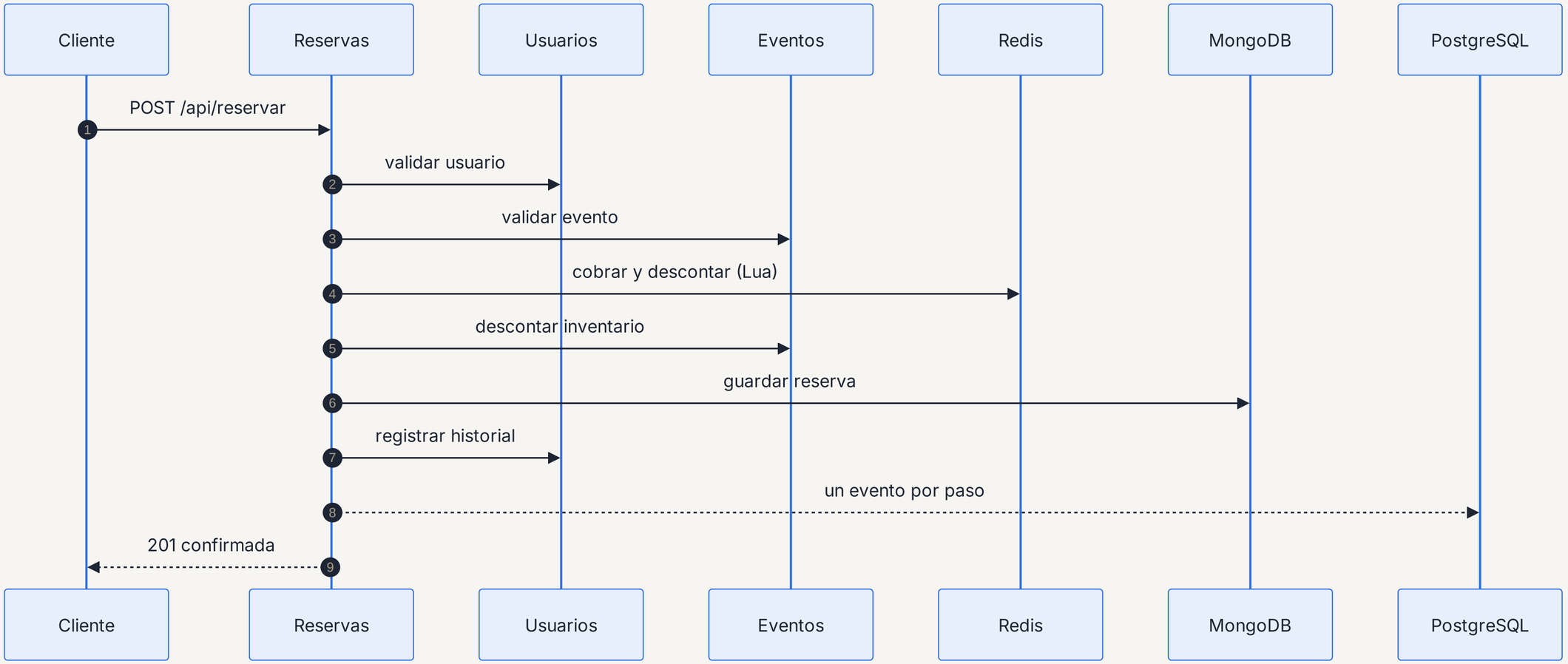
Seis handlers en serie: cada uno pasa o corta



Rojo: corta sin nada que deshacer · Naranja: ya cambió estado y lo revierte

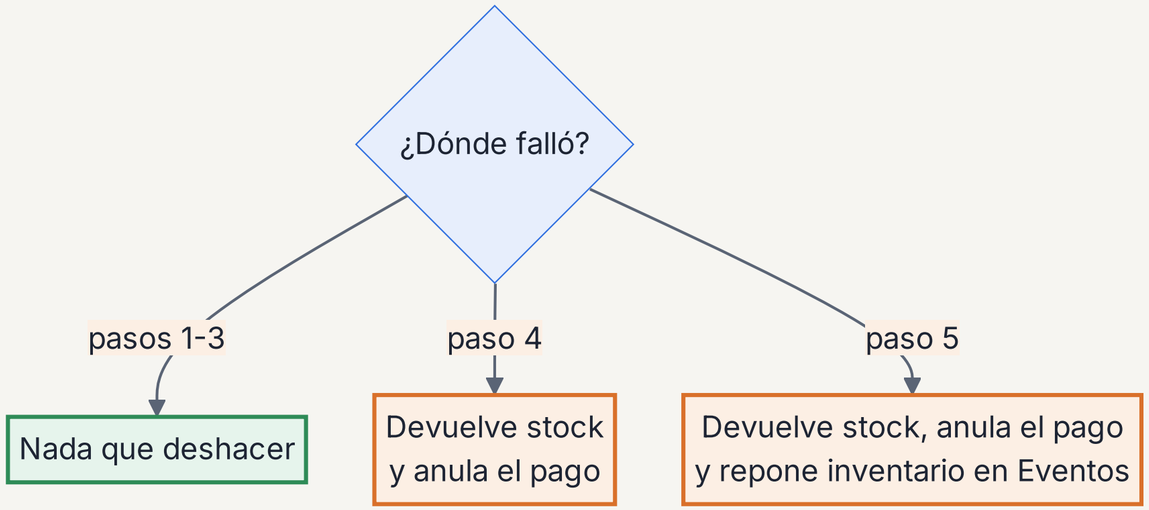
SAGA ORQUESTADA

Reservas coordina la compra de punta a punta



Orquestación: un solo lugar conoce el orden, así que compensar y auditar es directo

Si algo falla después de cobrar, se deshace



Auditoría real

inicio	SAGA_STARTED
+42 ms	PAGO_PROCESADO
+47 ms	COMPENSACION_EJECUTADA
+51 ms	SAGA_FAILED

Stock antes y después: 5 → 5.

Cada handler que cambia estado sabe deshacerlo

ATOMICIDAD

No hay doble venta: el stock lo decide un script Lua

```
-- Redis ejecuta un script por vez
if ya_pagado then
  return OK              -- idempotente
end
if disponible < cantidad then
  return SIN_STOCK       -- nada cambia
end
DECRBY stock cantidad
HSET   pago ...
return OK
```

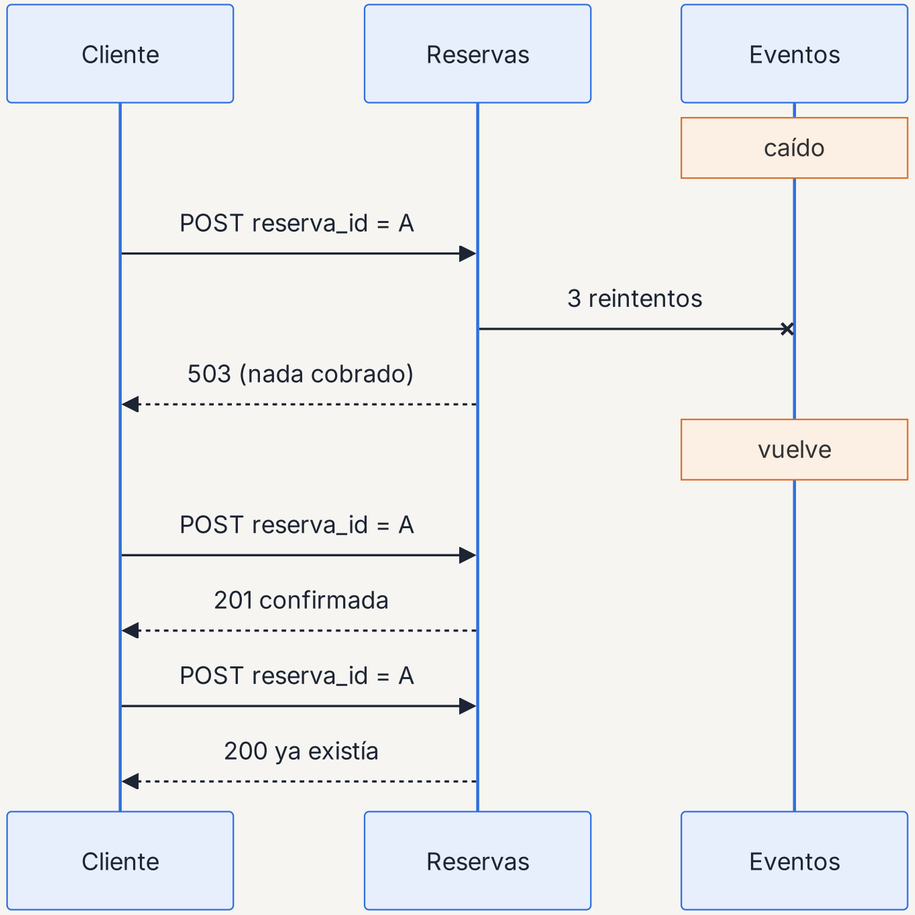
Prueba con JMeter

300 intentos contra 50 entradas

50	250	0
compras	rechazos	stock final

Chequear y descontar no pueden intercalarse entre dos compradores

Reintentar siempre es seguro



Mismo `reserva_id`, misma respuesta

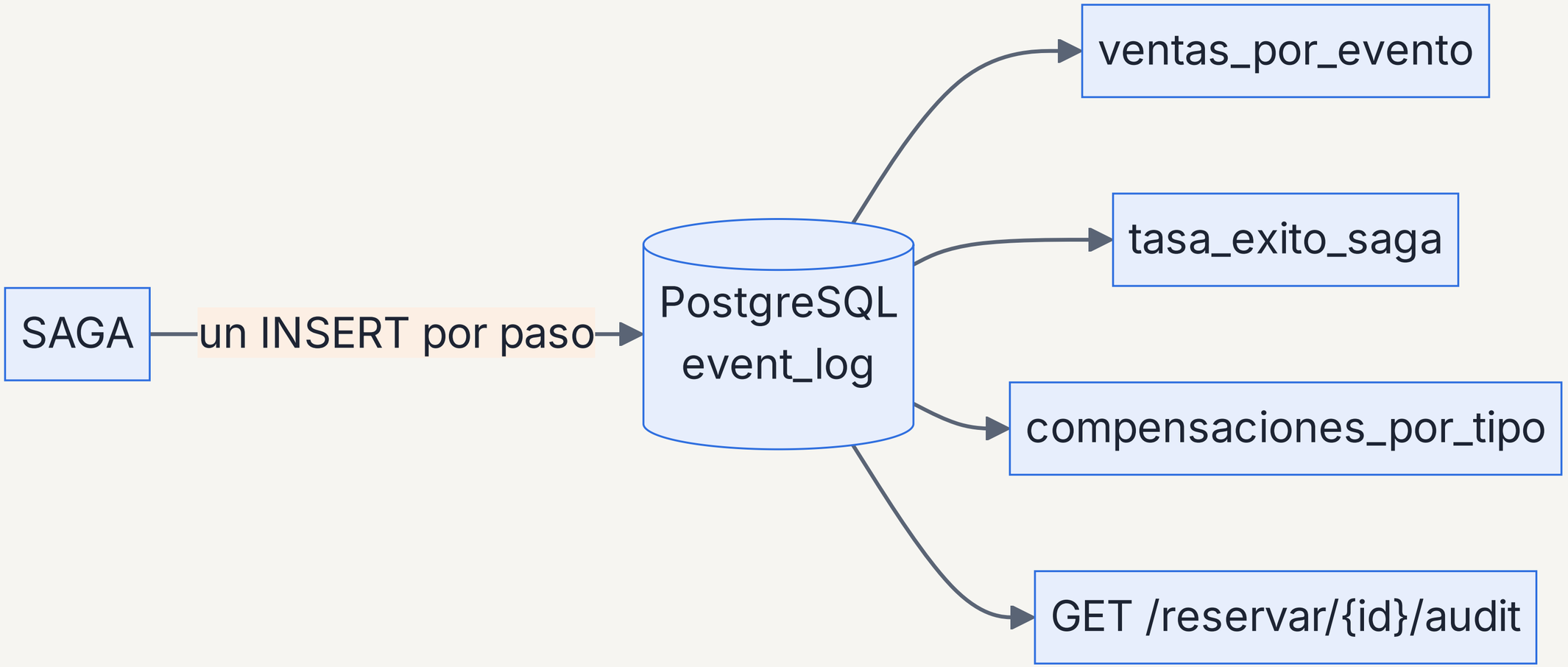
No se cobra ni se reserva dos veces.

Circuit breaker

Tras 5 fallos deja de insistir por 30 s.

AUDITORÍA

Event log append-only + CQRS



Se escribe aparte (command) y se consulta con vistas (query) · si PostgreSQL cae, la compra sigue

RESULTADOS

Qué está probado y qué falta

261
tests en verde

0
ventas de más bajo carga

≈50
req/s de techo (objetivo: 100)

Límites

- La compra satura en ≈ 50 req/s
- Un solo nodo por base, sin réplicas
- El event log es de auditoría: el estado no se reconstruye desde él

Detalle en jmeter/README.md y brain/learnings/learnings.md

CIERRE

Consistencia fuerte donde se mueve plata. Disponibilidad en el catálogo.

`make up` `make test` `make jmeter` `python3 demo/servidor.py`

Demo en vivo a continuación